

Segurança da plataforma — relatório consolidado

Plataforma VNH (Hype App) · estado em 3 de outubro de 2026

Consolida a auditoria de setembro/2026 (que teve acesso ao banco e ao painel do Supabase) e a auditoria técnica de outubro/2026 (análise do código, das migrations e do build, **sem** acesso ao banco vivo). Cada afirmação diz de onde vem: **[código]**, **[migrations]**, **[auditoria de setembro]** ou **[não verificado]**.

1. Resumo executivo

- **Base sólida.** RLS ligada nas 103 tabelas **[migrations + esquema vivo]**; login com MFA (TOTP), rate limit e auditoria de acessos **[código]**; todos os buckets de arquivos privados **[auditoria de setembro]**; webhooks e crons com segredo comparado em tempo constante e assinatura no webhook de e-mail **[código]**; HTML do blog sanitizado **[código]**.
- **Exposição principal em aberto (P1).** Contas do Portal do Cliente são usuários comuns (`authenticated`) no mesmo Supabase da equipe. Por falta de escopo em algumas policies e nas policies de storage, um cliente logado, chamando a API diretamente, **leria dados internos da agência e arquivos de outros clientes** (relatórios mensais, anexos financeiros). **[migrations — a confirmar no banco vivo]**. A correção está pronta e **não foi aplicada**.
- **Números.** Setembro: 11 achados (4 altos, todos corrigidos). Outubro: 1 achado P1 novo (pendente de aplicar), 4 correções no código, 1 endpoint legado removido.
- **O que depende de você:** (1) conferir o banco e aplicar a migration de RLS; (2) ligar "Leaked Password Protection" no painel do Supabase; (3) decidir CSP, rate limit de escrita pública, tipos MIME por bucket e proteção contra replay do webhook de leads.

2. Como o acesso funciona

Quem acessa e por onde

Quem	Superfície	Credencial	Validada em
Equipe interna	<code>App (/time , /clientes/\$id , /projeto/\$id , /chat-v2)</code>	E-mail + senha, MFA opcional	Supabase Auth + RLS
Cliente com login	Portal V2 (<code>/portal-v2/*</code>)	E-mail + senha por convite	Supabase Auth + server functions (service-role)
Cliente por link	Portal (<code>/portal/\$token/*</code>)	Token fixo do cliente	Server functions
Influenciador / lead / destinatário	Inscrição, NPS, bugs, proposta, descadastro	Token na URL	Server functions
Sistemas externos	Leads (Make/Typeform), Resend, Google, Vercel Cron	Segredo / assinatura / Bearer	Rotas <code>/api/*</code>

Autorização em camadas

1. **RLS no banco** — a barreira real. Funções: `is_admin` , `has_permission(uid, módulo)` , `is_internal_team_member` , `can_manage_problems` , `user_can_access_campanha` , `is_active_client_admin_of` .
2. **Permissões de módulo** (`profiles.permissions`): clientes, campanhas, contratos, projetos, influenciadores (+ bancário), reuniões, comercial, financeiro, time, metas, chat, configurações (+ subáreas), problemas. Todas as tabelas de domínio usam `has_permission` .
3. **Server functions:** sessão validada por `requiresSupabaseAuth` por padrão (132 de 160). As 28 sem sessão são 25 por token (seção 4) e 3 do fluxo de login (duas checagens de rate limit e o registro de falha).
4. **Interface:** esconde o que a pessoa não pode usar — conveniência, nunca a barreira.

Separação equipe x cliente

- Identidade interna ou de cliente vem de `organization_members` + `organizations.type` . Contas de cliente **não podem receber permissões** de módulo (trigger `prevent_client_permissions`) e o sistema impede remover ou rebaixar o **último administrador** de uma empresa-cliente.
- O Portal V2 acessa dados por server functions com service-role e valida papel (`client_standard` , `client_approver` , `client_viewer`) e campanha liberada. Com o cliente do usuário usa só `profiles` e o bucket `avatars` .
- Limite assumido: **não há isolamento por "workspace"** no sentido multi-tenant de SaaS; a fronteira é equipe x cada cliente.

3. Controles existentes

Área	Controle	Evidência
Sessão	JWT do Supabase validado (<code>getClaims</code>); sessão em IndexedDB; "manter conectado" desmarcado encerra ao fechar o navegador; troca de senha obrigatória no primeiro acesso	código
MFA	TOTP (<code>mfa.functions</code>), desafio exigido nas rotas internas e do portal; ativar/desativar vai para a auditoria	código
Rate limit	Login 10/15 min e recuperação 3/h por e-mail; convites 20/h; reenvio 1/45 s; verificação MFA 10/15 min; registro de falha de login 20/h	<code>rate-limit.server</code> (tabela <code>rate_limit_events</code>)
Recuperação de senha	E-mail do Supabase; ao concluir, encerra as outras sessões (<code>signOutOtherSessions</code>)	código
Auditoria	<code>access_audit_log</code> (login, falha, logout, troca de ambiente, convites, papéis, suspensão, MFA) e <code>settings_audit_log</code> , visíveis em Configurações	código
Cofre de senhas	Campos criptografados com AES-GCM; a chave mestra só sai por função de admin ou por concessão temporária (10 min) após código TOTP de um admin, com bloqueio após tentativas erradas	código
Webhooks e crons	Segredo comparado com <code>timingSafeEqual</code> ; webhook de leads com <code>X-Webhook-Secret</code> guardado em tabela (rotacionável); Resend com assinatura Svix; crons com <code>Bearer CRON_SECRET</code> ; OAuth do Google com <code>state</code> guardado e <code>APP_URL</code> canônico	código
Entrada de dados	Zod em todas as funções públicas; nomes de arquivo sanitizados; magic bytes no mídia kit de inscrição	código
XSS	Markdown do blog escapado e passado por DOMPurify (só <code>http(s)</code>); <code>dangerouslySetInnerHTML</code> em 4 pontos, os de conteúdo de usuário sanitizados	código
Erros	Portal e 4 arquivos públicos devolvem mensagem genérica (<code>throwSafeDbError</code>); erro real fica no log do servidor	código
Storage	6 buckets verificados como privados e com limite de tamanho em setembro; <code>avatars</code> e <code>bug-reports</code> têm policies por pasta/permissão e nenhuma policy para <code>anon</code> (o flag <code>public</code> desses dois não foi conferido); URLs assinadas para relatórios	auditoria de setembro + migrations
Cabeçalhos	<code>X-Content-Type-Options</code> , <code>X-Frame-Options: SAMEORIGIN</code> , <code>Referrer-Policy</code>	<code>vercel.json</code>
Segredos	Nenhum segredo no código nem no histórico do git; service-role só em <code>*.server.ts</code> /handlers	auditoria de setembro + código
Tokens públicos	<code>crypto.randomUUID()</code> sem hífen (122 bits), mínimo de tamanho antes de consultar o banco, posse revalidada por cliente (sem IDOR encontrado)	código

4. Superfícies públicas (sem sessão)

Entrada	Credencial	Proteção atual	Lacuna
<code>POST /api/public/leads</code>	<code>X-Webhook-Secret</code>	segredo em tempo constante, CORS aberto mas exige o segredo	sem proteção contra replay (R4)
<code>/inscricao/\$token</code>	token da campanha	zod, idempotência, magic bytes no anexo	escrita sem rate limit
<code>/nps-influenciador/\$token</code>	token do NPS	zod, erro genérico	escrita sem rate limit
<code>/bugs/\$token</code>	token do projeto	zod, erro genérico, upload em <code>public/<token>/</code>	escrita e upload sem rate limit
<code>/calculadora-proposta/\$token</code>	token do lead	zod, erro genérico	escrita sem rate limit
<code>/portal/\$token/*</code>	token do cliente	posse revalidada, erro genérico	sem rate limit
<code>/email/descadastro/\$token</code>	token do destinatário	zod	—
<code>/api/webhooks/resend</code>	assinatura Svix	verificação HMAC	—
<code>/api/cron/*</code> , <code>/api/google/oauth-callback</code>	<code>Bearer / state</code>	segredo / estado guardado	—
Login e recuperação	e-mail + senha	rate limit consultivo (chamado pelo cliente antes do Supabase Auth)	quem chama o Supabase Auth direto ignora; vale o limite nativo do Supabase

5. Achados

5.1 Setembro/2026 (auditoria com acesso ao banco)

#	Sev.	Achado	Status hoje
1	Alta	XSS armazenado no Markdown do blog (chegava ao Portal)	Corrigido
2	Alta	<code>campanha_cronograma</code> aberta a qualquer autenticado	Corrigido
5	Alta	Mensagens de canais privados legíveis por qualquer autenticado	Corrigido
5b	Alta	RPCs <code>SECURITY_DEFINER</code> do Chat sem checar a conversa	Corrigido
3	Média	5 de 6 buckets sem limite de tamanho	Corrigido
4	Média	Sem cabeçalhos de segurança	Corrigido sem CSP
6	Média	Sem rate limit/CAPTCHA em endpoints públicos	Parcial: login, recuperação, convites e MFA têm; escrita pública por token não
7	Baixa/Média	Webhook de leads sem proteção contra replay	Aberto (regra de negócio)
8	Baixa	<code>hypito_payload</code> forjável	Corrigido
9	Baixa	<code>author_id IS NULL</code> permite mascarar autoria como "sistema"	Risco aceito
10	Baixa	Funções do Hypito liam antes de checar permissão	Corrigido
—	Info	"Leaked Password Protection" desligado no Auth	Aberto (painel do Supabase)
—	Info	Sem restrição de tipo MIME por bucket	Aberto (decisão de produto)

5.2 Outubro/2026

Sev.	Achado	Fonte	Status
P1	Policies <code>T0 authenticated USING (true)</code> e policies de storage só com <code>bucket_id</code> expõem a contas de cliente: precificação (margem/comissão), estado compartilhado (leitura e <code>UPDATE</code> de qualquer linha), tags, AEO, e-mails de campanhas, dados de desempenho, interações do blog; e nos buckets <code>relatorios-mensais</code> , <code>financeiro-anexos</code> , <code>entrega-anexos</code> , <code>aeo-evidencias</code> , <code>task-attachments</code> , listagem e download (e upload)	migrations, em ordem	Migration pronta, não aplicada
P2	Endpoint legado sem sessão (<code>requestPasswordReset</code>) gravava no banco e enviava push aos admins para qualquer chamador	código	Removido
P2	Registro de falha de login sem sessão e sem teto (inundava o log de um e-mail alheio)	código	Rate limit 20/h
P2	Erros de banco crus devolvidos em 4 arquivos públicos (18 pontos)	código	Genéricos
P2	Rate limit de login consultivo; escrita pública por token sem limite	código	Aberto
P3	<code>dompurify</code> com advisory (3.4.14); o código não usa <code>IN_PLACE</code> /hooks	<code>bun audit</code>	Atualizado para 3.4.16
P3	Cópia obsoleta e menos segura de <code>portal-auth.functions.ts</code> no repositório	código	Removida
P3	10 tabelas com RLS e nenhuma policy (vault, webhooks, rate limit, Google, config de e-mail)	migrations	Intencional (só service-role)
P3	<code>access-guards.server.ts</code> e <code>hypito-permissions.server.ts</code> : guardas escritos e nunca ligados	grafo de imports	Decidir: adotar ou remover

6. Pendências, em ordem

Prio.	Ação	Quem	Onde
P1	Rodar as 3 consultas de verificação no SQL Editor; se a 2ª vier vazia, aplicar a migration <code>20261004000000</code> e testar como equipe e como cliente. Rollback pronto	você + dev	<code>rls-internal-only.md</code>
P2	Ligar Leaked Password Protection (Authentication → Policies)	você	painel do Supabase

Prio.	Ação	Quem	Onde
P2	Rate limit no servidor para escrita pública por token (inscrição, NPS, bugs, proposta, comentários)	dev	<code>*.functions.ts</code> públicos
P2	Conferir os limites nativos de rate limit do Supabase Auth (piso real do login)	você	painel do Supabase
P2	CSP em modo <code>Report-Only</code> por alguns dias antes de impor (risco de quebrar scripts/estilos)	dev	<code>vercel.json</code>
P2	Tipos MIME por bucket (levantar o que cada fluxo usa) + magic bytes nos uploads	dev + produto	buckets
P2	Proteção contra replay no webhook de leads (chave de idempotência do Make/Typeform ou timestamp assinado)	dev + produto	<code>api/public/leads</code>
P3	Decidir o destino de <code>access-guards / hypito-permissions</code> ; das 10 funções <code>*Session</code> do portal V2 sem tela	produto	<code>src/lib</code>
P3	Atualizar dependências de build quando liberadas	dev	<code>bun update</code>

7. Limites do desenho (aceitos ou a conhecer)

- **Sessão em IndexedDB**: legível por qualquer JavaScript da página; um XSS amplifica o dano (por isso a sanitização do blog é crítica).
- **JWT validado criptograficamente**, mas não se confere a cada chamada se o usuário ainda está ativo.
- **Chave mestra do cofre** fica na tabela `vault_secret` (só service-role), no mesmo banco dos dados cifrados: quem tiver acesso ao banco com service-role lê chave e dados.
- `author_id IS NULL` no Chat permite mascarar autoria como "sistema" (risco baixo, aceito).
- Permissões `membros` e `senhas` são **decorativas** (a barreira real é admin e `configuracoes:senhas`), para impedir auto-promoção a admin.
- **Hypito** roda fora deste repositório; a auditoria de setembro (que viu o código dele) não encontrou integração com LLM nem risco de prompt injection, e confirmou ações em duas etapas com permissão revalidada. Hoje esse código não está no repositório, então isso não foi reconferido.

8. O que foi e o que não foi verificado

Verificado	Como
Políticas efetivas, funções, índices, buckets	Migrations aplicadas em ordem + <code>types.ts</code> (esquema vivo, 103 tabelas, sem drift)
Autorização das server functions	Varredura de 160 funções: middleware, zod, rate limit, uso de service-role
XSS, segredos, tokens, headers, dependências	Leitura de código, <code>bun audit</code> , <code>vercel.json</code>
Regressão	<code>tsc</code> , ESLint (0 erros), 734 testes, build (todos passando em 2026-10-03)

Não verificado	Impacto
Banco vivo (políticas reais, dados)	A exposição P1 vem das migrations; precisa de confirmação
Teste dinâmico/penetração contra produção	Nenhuma tentativa real com conta de cliente
Painel do Supabase (limites de Auth, Leaked Password Protection)	Configuração fora do repositório
CSP, LGPD ponta a ponta	Fora do escopo das duas rodadas

Consultas de verificação (SQL Editor do Supabase; texto completo, teste e rollback em `rls-internal-only.md`): 1. Políticas "abertas": `select tablename, policyname, cmd, roles from pg_policies where schemaname in ('public','storage') and (qual = 'true' or with_check = 'true');` 2. Quem perderia acesso após a migration (esperado: nenhuma linha): perfis que não são admin, nem membros de organização interna, nem de organização de cliente. 3. Para cada usuário interno real: `is_internal_team_member(id)` e `is_admin(id)`.

9. Dependências

`bun audit`: 28 vulnerabilidades (18 altas, 8 moderadas, 2 baixas), todas em ferramentas de **build, lint e teste** (vite/postcss/nanoid, babel, esbuild, browserslist, js-yaml via ESLint e plugin do TanStack, vitest), segundo o caminho de dependência reportado — nenhuma em código que roda no navegador. A única dependência de runtime com advisory (`dompurify`) foi atualizada.

10. Histórico

Data	O que aconteceu
2026-08-30	Fechamento de RLS por permissão nas tabelas de domínio
2026-09-17	Auditoria de segurança completa; correções de XSS, cronograma, chat, buckets, headers
2026-09-18	MFA, recuperação de senha autoatendida, rate limit em Postgres
2026-09-28	Separação equipe x cliente (perfis, chat, desempenho)
2026-10-01	Trigger que impede permissões em contas de cliente
2026-10-03	Auditoria técnica: endpoint legado removido, rate limit no log de falhas, erros genéricos, dompurify, migration de RLS interna (pendente)

Documentos de apoio: security/README.md (estado resumido), 2026-09-security-audit-report.md , 2026-09-security-remediation-plan.md , rls-internal-only.md .